



V A S D P

The **V**oice **A**ctivated **S**peaker for **D**isabled **P**eople

Davi Volpi Takata Zevallos 301372653

Jaturaput Jongsubcharoen 301391425

Kripa Acharya 301432879

Ratthapat Apichaikunlanit 301275267

Revision Table	4
PROJECT PART A	5
Section 1 “Introduction” and sub sections:	5
1.1 Purpose:	5
1.1.1 User Stories for "Voice Activated Speaker for Disabled People (VASDP)":	5
1.2 Document Conventions:	6
1.2.1 Formatting Conventions	6
1.2.2 Table of Acronyms used in the document:	6
1.2.3 Terminology	7
1.3 Intended Audience and Reading Suggestions:	7
1.4 Project Scope:	8
1.5 References:	8
Section 2 “Overall description” and sub sections:	9
2.1 Product Perspective:	9
2.2 Product Features (Functions):	10
2.3 User classes and Characteristics:	10
2.4 Operating Environment:	10
2.7 Assumptions and Dependencies:	10
Section 3 “External Interface Requirements”	11
3.1 User Interfaces:	11
3.2 Hardware Interfaces:	12
3.3 Software Interfaces:	12
3.4 Use Cases Table	13
Subsystem: User Management Subsystem	13
Subsystem: Voice Command Processing Subsystem	13
Subsystem: Integration and Configuration Subsystem	14
Subsystem: System Maintenance Subsystem	14
Section 4	16
4.1 Functional Requirements list	16
4.2 Use Case Formal Descriptions	18
4.3 SwimLane Activity Diagram	20
4.4 The Voice Activated Speaker for Disabled People System	21
4.5 User Case Diagram	22
Section 5	24
5.1 Nonfunctional Requirements list	24
APPENDIX C	26
Stakeholder Register	26
APPENDIX D	27
Interview Questions	27
APPENDIX E	30
CLASS DIAGRAM	30
1. User	30
2. ProfileManager	30
3. VoiceCommandProcessor	30

4. TaskExecutor	31
5. DeviceIntegrator	31
6. SecurityManager	32
7. SystemHealthMonitor	33
8. AccessibilitySettings	33
UML static structure diagram:	35
Class Relationships	35
CRC Cards for "Voice Activated Speaker for Disabled People"	36
1. Class: User	36
2. Class: ProfileManager	38
Class: ProfileManager	38
3. Class: VoiceCommandProcessor	38
Class: VoiceCommandProcessor	39
4. Class: TaskExecutor	39
Class: TaskExecutor	40
5. Class: DeviceIntegrator	40
Class: DeviceIntegrator	40
6. Class: SecurityManager	41
Class: SecurityManager	41
7. Class: SystemHealthMonitor	42
Class: SystemHealthMonitor	42
8. Class: AccessibilitySettings	43
Class: AccessibilitySettings	43
Updated First Cut Domain Class Diagram	44
a. Class Attribute Descriptions and Visibilities	44
b. Relationships and Signature Methods	46
Abstract State Machine Diagram for TaskExecutor	48
Abstract State Machine Diagram for VoiceCommandProcessor	49
Abstract Sequence Diagram for Voice Command Processing	51
Sequence Diagram: Voice Command Processing	53

Revision Table

Version	Description	Name	Date	Approved by
1.0.0	Initializing the structure of the project from section 1 to 3.	Jaturaput J.	2024/05/22	Davi Z.
1.1.0	Added the Product Perspective diagram.	Jaturaput J.	2024/05/22	Rathapat A.
1.2.0	Removed the template material.	Davi Z.	2024/05/25	Rathapat A.
1.2.0	Designed The Voice Activated Software Speaker for Disabled People (VASDP) logo	Jaturaput J.	2024/05/26	Rathapat A.
1.2.1	Added and improving Functional Requirements list and Non-Functional Requirements list	Jaturaput J.	2024/05/26	Kripa A.
1.2.2	Added a low interest of stakeholder in Stakeholder Register table	Rathapat A.	2024/05/26	Kripa A.
1.2.3	Edited Software Interface by moving Smart Home Devices from Hardware Interfaces into Software Interface	Kripa A.	2024/05/27	Davi Z.
1.2.4	Added table of content and added the User Interfaces diagram.	Davi Z.	2024/05/27	Kripa A.
1.3.0	Added the Interview Questions.	Davi Z.	2024/05/29	Davi Z.
2.0.0	Revise Assignment(Part A) following the feedback “2024/06/12”	Rathapat A.	2024/06/12	Jaturaput J.
2.1.0	Create the Swimlane diagram and Usercase diagram	Jaturaput J.	2024/06/17	Davi Z.
2.2.1	Change the name of the use case along with system from the feedback “2024/06/24”	Jaturaput J.	2024/06/28	Rathapat A.
2.2.2	Move the actors out of the system box following the feedback “2024/06/24” .	Jaturaput J.	2024/06/28	Kripa A.
2.2.2	update the swimlane abstract “Revising Main Success Scenario” following the feedback “2024/06/24” .	Kripa A.	2024/06/29	Jaturaput J.
2.3.1	Create a list of analysis classes related to the domain problem/opportunity of your project.	Jaturaput J.	2024/06/30	Davi Z.

2.3.2	Create a UML static structure diagram	Jaturaput J.	2024/07/01	Kripa A.
2.3.3	Create the CRC card (front side)	Jaturaput J.	2024/07/02	Rathapat A.
2.3.4	Update First cut Domain Class Diagram	Jaturaput J.	2024/07/08	Davi Z.
2.4.0	Update the Swimlane Diagram "Adding decision symbol" following the feedback "2024/07/10" .	Jaturaput J.	2024/07/10	Rathapat A.
2.4.1	Update the User Case Diagram following the feedback "2024/07/10" .	Rathapat A.	2024/07/10	Davi Z.
2.4.2	Update the UML static structure "Adding the Aggregation & composition" following the feedback "2024/07/10" .	Jaturaput J.	2024/07/11	Kripa A.
2.4.3	Create the CRC card (back side)	Jaturaput J.	2024/07/11	Davi Z.
2.4.4	Update the Revision table of the Section 4 and Section 5	Davi Z.	2024/07/11	Rathapat A.
2.4.5	Update the Revision table and added Appendix C, D, E	Davi Z.	2024/07/11	Kripa A.
3.1.1	Create the Abstract State Machine Diagram and State Machine Diagram	Jaturaput J.	2024/07/22	Kripa A.
3.2.1	Create the Abstract of Sequence Diagram and Sequence Diagram	Jaturaput J.	2024/07/23	Davi Z.
3.3.1	Create the second State Machine Diagram	Kripa A.	2024/08/06	Jaturaput J.
3.3.2	Revise the Sequence Diagram	Davi Z.	2024/08/06	Kripa A.
3.4.1	Prepare the slides for presentation	Davi Z.	2024/08/06	Jaturaput J.
3.5.1	Revise the State Machine Diagram from Prof. feedback by adding an ending point.	Jaturaput J.	2024/08/07	Rathapat A.
3.5.2	Revise the Sequence Diagram from Prof. feedback by removing a unnecessary if else condition	Jaturaput J.	2024/08/07	Davi Z.
3.6.1	Update the Revision table and added Appendix E in the Document file.	Jaturaput J.	2024/08/07	Rathapat A.

3.7.1	Revise the Slides by Changing the image of State Machine Diagram and Sequence Diagram	Rathapat A.	2024/08/08	Jaturaput J.
-------	--	-------------	------------	--------------

Section 1 “Introduction” and sub sections:

1.1 Purpose:

This new software product will address the problem of Disabled people except deaf people with the voice activated speaker and will be integrated into the infrastructure of Canada, this is version 1.0.0.

1.1.1 User Stories for "Voice Activated Speaker for Disabled People (VASDP)":

1. **As a Disabled User, I want to create and manage my profile, so that I can have a personalized experience with the VASDP.**
 - a. **Acceptance Criteria:**
 - i. The system should allow the creation of a user profile with personalized settings.
 - ii. The user should be able to update their profile information at any time.
2. **As a Disabled User, I want to issue voice commands to control my smart home devices, so that I can perform tasks independently.**
 - a. **Acceptance Criteria:**
 - i. The system should recognize and process voice commands accurately.
 - ii. The system should provide feedback confirming the command or executing the requested action.
3. **As a Caregiver, I want to assist in setting up and managing the VASDP for the disabled users, so that they can have a seamless experience.**
 - a. **Acceptance Criteria:**
 - i. The system should allow caregivers to manage user settings and preferences.
 - ii. The system should enable caregivers to troubleshoot and resolve common issues.
4. **As a System Administrator, I want to monitor and maintain the system health, so that the VASDP remains reliable and performs well.**
 - a. **Acceptance Criteria:**
 - i. The system should provide real-time monitoring of performance metrics.
 - ii. The system should generate health reports and alert administrators of any issues.
5. **As an IT Security Officer, I want to ensure the security of user data, so that all personal information is protected and secure.**
 - a. **Acceptance Criteria:**
 - i. The system should implement robust encryption and authentication protocols.

- ii. The system should regularly update security measures and conduct audits.
 - 6. **As a User, I want to set and customize my accessibility settings, so that the VASDP meets my specific needs.**
 - a. **Acceptance Criteria:**
 - i. The system should allow users to adjust visual and auditory settings.
 - ii. The system should save and apply these settings across sessions.
 - 7. **As a Developer, I want to integrate VASDP with existing smart home devices, so that it can control a wide range of devices.**
 - a. **Acceptance Criteria:**
 - i. The system should be compatible with major smart home ecosystems like Google Home and Amazon Alexa.
 - ii. The system should handle the addition and removal of devices seamlessly.
-

1.2 Document Conventions:

1.2.1 Formatting Conventions

1. **Headings and Subheadings:**
 - Major sections are titled in bold and numbered (e.g., **1. Introduction**).
 - Subsections are also numbered and titled in bold (e.g., **1.1 Purpose**).
2. **Text Styles:**
 - **Bold Text:** Used for section titles, important terms, and key concepts.
 - *Italic Text:* Used for emphasis and for the titles of documents, books, articles, and other publications.
3. **Tables:**
 - Tables are used to organize and present data clearly. Each table has a title and is numbered sequentially (e.g., **Table 1: List of Acronyms**).

1.2.2 Table of Acronyms used in the document:

Acronyms	Description
SQL	Sequential query language
DB	Database

<i>DBMS</i>	Database management System
<i>API</i>	Application Programming Interface
<i>UI</i>	User Interface
<i>VRS</i>	Voice Recognition System
<i>SDK</i>	Software Development Kit
<i>VASDP</i>	Voice Activated Speaker for Disabled People

1.2.3 Terminology

1. **User:** Refers to the disabled individuals utilizing the VASDP for their daily tasks.
2. **System Administrator:** Personnel responsible for the maintenance and security of the VASDP system.
3. **Caregiver:** Individuals assisting disabled users with the setup and management of the VASDP.

1.3 Intended Audience and Reading Suggestions:

Intended Audience:

1. Developers: Responsible for coding and implementation of the software.
2. Project Managers: Overseeing the project progress, timelines, and resource allocation.
3. Marketing Staff: Planning and executing marketing strategies for the software.
4. End Users: Disabled individuals who will use the voice-activated speaker.
5. Testers: Conducting various tests to ensure software quality and functionality.
6. Documentation Writers: Creating user manuals, help guides, and other documentation for the software.

Reading Suggestions:

1. Developers: Focus on sections detailing technical specifications, API, and SDK integration.
2. Project Managers: Concentrate on the project scope, objectives, milestones, and timeline.
3. Marketing Staff: Review the benefits, objectives, goals. Focus on user cases and product features.
4. End Users: Pay attention to the user interface design, features, and usability aspects.
5. Testers: Examine the detailed requirements, test cases, and quality assurance measures.
6. Documentation Writers: Reference the comprehensive details provided in each section to create accurate and helpful documentation.

1.4 Project Scope:

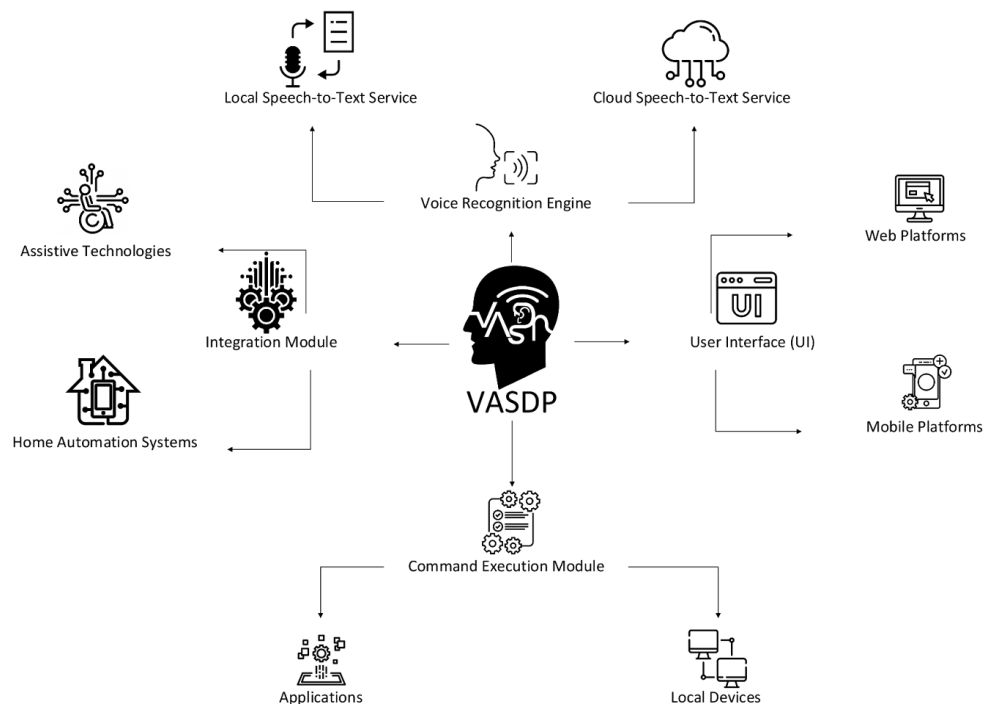
The Voice Activated Software Speaker for Disabled People (VASDP) aims to enhance accessibility for individuals with disabilities (excluding the deaf) by providing a voice-controlled interface for various tasks. This software will integrate seamlessly into Canada's existing technological infrastructure, offering a user-friendly solution to improve independence and quality of life for the disabled community.

1.5 References:

1. Accessibility for Ontarians with Disabilities Act (AODA)
Matthew, C. (2016, April 4th) *The Accessibility for Ontarians with Disabilities Act: A Refresher for Nonprofits* [Website]. Retrieved on May 19th 2024, https://www.techsoup.ca/community/blog/the-accessibility-for-ontarians-with-disabilities-act-a-refresher-for-nonprofits?qad_source=1&qclid=Cj0KCQjw6auyBhDzARIsALIo6v8gnzEB1dXGqzpHFQWq03HJk_z3oU3yNfJrp3FARIXDS-xZrJDUQ8aAnUVEALw_wcB
2. Voice Recognition Technology in Assistive Devices
Missy, J. (2023, March 2nd) *Types of Assistive Technology Tools* [Website]. Retrieved on May 19th 2024, <https://www.audioeye.com/post/types-of-assistive-technologies/>
3. Integrating Assistive Technology into Existing Infrastructure
Bijleveld, P. (2023, December 3rd) *ASSISTIVE TECHNOLOGY DESERVES OUR URGENT ATTENTION* [Website]. Retrieved on May 19th 2024, <https://www.unops.org/news-and-stories/insights/assistive-technology-deserves-our-urgent-attention>
4. Market Trends in Assistive Technology
pharmiweb. (2024, April 22nd) *Assistive Technology Market Expected to Exceed USD 36.6 Billion by 2033* [Website]. Retrieved on May 19th 2024, <https://www.pharmiweb.com/press-release/2024-04-22/assistive-technology-market-expected-to-exceed-usd-366-billion-by-2033>

Section 2 “Overall description” and sub sections:

2.1 Product Perspective:



Context and Origin:

The Voice Activated Software Speaker for Disabled People (VASDP) is a new, self-contained software product designed to enhance accessibility for disabled individuals, excluding those who are deaf. This software aims to provide a voice-activated interface to perform various tasks, improving the independence and quality of life for its users.

Relationship to Other Systems:

While VASDP is a standalone product, it will integrate with existing infrastructure and technologies within Canada. It will interface with various external systems and devices to ensure seamless functionality. The integration points include:

- Voice Recognition Engine: Interfaces with cloud-based or local speech-to-text services.
- User Interface (UI): Compatible with standard web and mobile platforms.
- Command Execution Module: Interacts with local devices and applications to execute commands.
- Integration Module: Connects with other assistive technologies and home automation systems.

2.2 Product Features (Functions):

The VASDP software will provide the following major functionalities:

1. Voice Command Recognition: Accurately interpret and respond to user voice commands.
2. Task Execution: Perform tasks such as setting reminders, controlling home devices, and accessing information.
3. User Management: Allow users to create and manage profiles.
4. Accessibility Settings: Customize the interface and commands to suit individual user needs.
5. Integration with External Systems: Connect with other devices and applications for extended functionalities.

2.3 User classes and Characteristics:

The primary user classes for VASDP include:

1. Disabled Individuals: The main users who will use the software for daily tasks through voice commands.
2. Non-Disabled Individuals: users who have interest in the software for the voice command software.
3. Caregivers: Assist disabled users in setting up and managing the software.
4. System Administrators: Responsible for maintaining the software, managing updates, and ensuring system security.

2.4 Operating Environment:

VASDP will operate in the following environments:

1. Hardware Platform: Compatible with standard desktop, mobile, microphone and smart home devices.
2. Operating System: Supports Windows, macOS, iOS, Android, and Linux.
3. Additional Software Components:
 - 3.1 Uses cloud-based speech-to-text services (e.g., Google Cloud Speech-to-Text, Amazon Transcribe).
 - 3.2 Supports web browsers including Firefox, Chrome, Safari, and Edge.
 - 3.3 Integrates with smart home ecosystems like Google Home and Amazon Alexa.

2.7 Assumptions and Dependencies:

Assumptions:

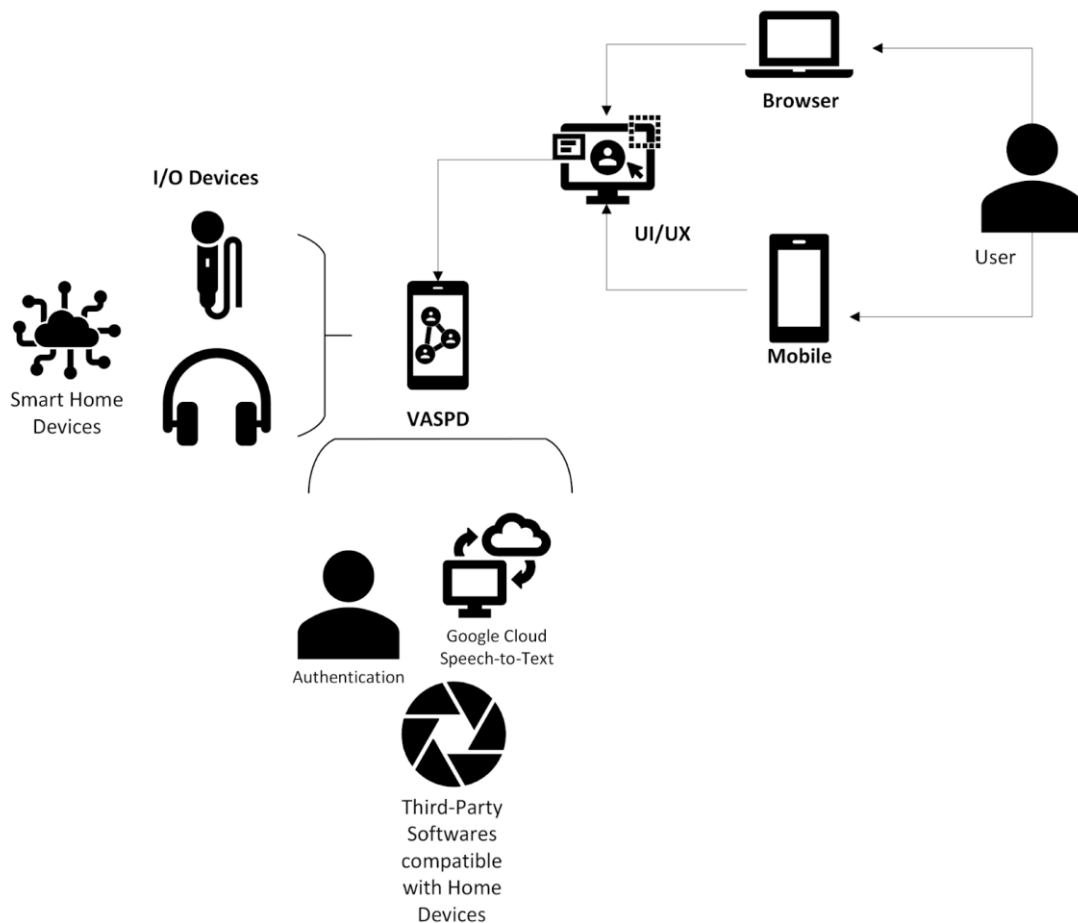
1. Users have access to compatible devices with internet connectivity.
2. The voice recognition service has high accuracy for various accents and speech patterns.
3. Users will have basic technical knowledge to operate voice-activated systems.

Dependencies:

1. Third-Party Services: Reliance on cloud-based voice recognition services.
2. Development Environment: Development tools and libraries compatible with multiple operating systems.
3. Integration Points: Compatibility with existing smart home devices and infrastructure.
4. Regulatory Compliance: Adherence to privacy and data protection laws applicable in Canada.

Section 3 “External Interface Requirements”

3.1 User Interfaces:



Overview:

End users will access the Voice Activated Software Speaker for Disabled People (VASDP) through multiple interfaces, ensuring flexibility and convenience. These interfaces include a mobile app and a web interface.

Mobile Interface:

1. Supported Platforms: iOS and Android.
2. Features: Users will be able to interact with the software using voice commands, configure settings, manage profiles, and access task execution functionalities.
3. Development Technologies: The mobile app will be developed using cross-platform frameworks such as Flutter or React Native, ensuring consistent performance across iOS and Android devices.

Web Interface:

1. Supported Browsers: Firefox, Chrome, Safari, and Edge.
2. Features: Similar to the mobile app, the web interface will allow users to issue voice commands, configure settings, manage profiles, and execute tasks.
3. Development Technologies: The web interface will be built using modern web technologies such as HTML5, CSS3, and JavaScript (React.js framework). The backend will utilize Node.js and Express for handling requests and integrating with the voice recognition service.

3.2 Hardware Interfaces:

VASDP will interface with various hardware devices to provide comprehensive voice-activated functionalities. The key hardware interfaces include:

1. Microphones:

Purpose: Capture voice commands from the user.

Integration: Built-in microphones in mobile devices, external microphones for desktop and home automation systems.

2. Speakers:

Purpose: Provide audio feedback and responses to user commands.

Integration: Built-in speakers in mobile devices, external speakers for desktop and home automation systems.

3. Cameras (Optional):

Purpose: Enable additional functionalities such as visual recognition or video calls (future versions).

Integration: Built-in cameras in mobile devices and webcams for desktop systems.

3.3 Software Interfaces:

Overview:

This subsection outlines the third-party software components that the Voice Activated Software Speaker for Disabled People (VASDP) will interact with. These software interfaces are critical for providing essential functionalities such as voice recognition, user authentication, and smart home device integration.

1. Voice Recognition Service:

- Third-Party Software: Google Cloud Speech-to-Text
- Version: Latest stable release
- Purpose: Convert user voice commands into text for processing
- Reference: Google Cloud Speech-to-Text Documentation

2. User Authentication:

- Third-Party Software: Auth0
- Version: Latest stable release
- Purpose: Secure user authentication and authorization
- Reference: Auth0 Documentation

3. Smart Home Integration:

- Third-Party Software: Amazon Alexa Skills Kit (ASK)
- Version: Latest stable release
- Purpose: Interface with Amazon Alexa for smart home control
- Reference: Amazon Alexa Skills Kit Documentation

4. Smart Home Devices:

- Control home automation systems (e.g., lights, thermostats, security systems) through voice commands.
- Interfaces with devices such as Google Home, Amazon Alexa, and smart plugs via their respective APIs.

3.4 Use Cases Table

Subsystem: User Management Subsystem

Use Case Name	List of Related Requirements ID	Actor(s)	Brief Description
Create-User Profile	All of Requirement ID	User, Caregiver	The actor creates a new user profile with personalized settings, including visual adjustments and voice command preferences.
Manage-User Settings	All of Requirement ID	User, Caregiver	The actor updates user settings such as voice command preferences, color themes, and accessibility options.

Subsystem: Voice Command Processing Subsystem

Use Case Name	List of Related Requirements ID	Actor(s)	Brief Description
Recognize-Voice Command	All of Requirement ID	User	The system accurately recognizes and processes voice commands issued by the user, translating them into actions or responses.

Execute-Task	All of Requirement ID	User, Caregiver	The system performs tasks such as setting reminders, controlling home devices, and accessing information based on recognized voice commands.
--------------	-----------------------	-----------------	--

Subsystem: Integration and Configuration Subsystem

Use Case Name	List of Related Requirements ID	Actor(s)	Brief Description
Configure-Integration	FR012, NFR04	System Administrator	The actor configures the system to integrate with various external devices and services such as smart home systems and other assistive technologies.
Manage-Security	NFR02	IT Security Officer	The actor ensures that all user data is encrypted and secure, configuring necessary security measures such as authentication and authorization protocols.
Ensure-Compatibility	NFR04, NFR07	System Administrator	The actor verifies and manages the system's compatibility with different hardware platforms, operating systems, and third-party services.

Subsystem: System Maintenance Subsystem

Use Case Name	List of Related Requirements ID	Actor(s)	Brief Description
Monitor-System Health	NFR03, NFR05	IT Administrator	The actor monitors system performance and uptime, ensuring the system is available and scales according to user demand.
Update-System	NFR01, NFR08	System Administrator	The actor updates the system with new features, security patches, and performance improvements to ensure continuous improvement in accessibility and functionality.

Section 4

Deliverable #2

4.1 Functional Requirements list				
Requirement ID	Requirement Title	Short Description	Priority	Requester
FR01	Amputation Support	The system should allow voice-only operation for users with limb amputations, including setting up and managing the device without physical interaction.	High	Amputees
FR02	Repetitive Stress Injury Support	The system should minimize repetitive actions by enabling voice commands for	High	Repetitive stress injury user

		all functionalities to reduce physical strain.		
FR03	Color Blindness Adaptations	The system should support color-blind-friendly modes with high-contrast and customizable color schemes for visual feedback.	High	Colorblind user
FR04	Autism Spectrum Disorder Support	The system should include predictable, consistent responses and customizable settings to accommodate sensory sensitivities.	Normal/Medium	Austistic user
FR05	Low Vision Support	The system should provide enlarged text and icons on any connected display and robust voice feedback for navigation.	High	Low vision user
FR06	Blindness Support	The system should offer comprehensive voice guidance for setup and operation, with all interactions being voice-based.	High	Blind user
FR07	Memory Impairments Support	The system should include reminders and notifications with detailed verbal descriptions to	Normal/Medium	Memory impairments user

		assist users with memory impairments.		
FR08	Voice Training Module	The system should include a voice training module to better recognize the unique speech patterns of users over time.	Low	Dysarthria user
FR09	Color Theme Personalization	The system should offer a wide range of color themes to be set by users, helping those with visual preferences or needs.	Low	Colorblind user
FR010	Customizable Wake Words	The system should allow users to customize the wake word to any preferred phrase for better accessibility.	Expected	General User
FR011	Voice Command Recognition	The system should accurately recognize and respond to voice commands from users with various speech patterns and disabilities.	Expected	General User
FR012	Integration with Assistive Devices	The system should include features like voice control for text-to-speech	Expected	General User

		and speech-to-text functionalities.		
--	--	-------------------------------------	--	--

4.2 Use Case Formal Descriptions

Use Case Name: Recognize-Voice Command

Primary Actor: User (Disable people)

Stakeholders and Interests:

- **User** (Disable people) : Wants to interact with the system using voice commands for ease of use.
- **Caregiver:** Wants to ensure the system accurately recognizes commands to assist the user.
- **System Administrator:** Needs to ensure the system's voice recognition functions reliably and efficiently.

Preconditions:

1. The user is logged into the system.
2. The voice recognition system is initialized and running.
3. The microphone is functioning and properly configured.
4. The camera is properly set up and functioning if required.

Postconditions:

1. The system correctly interprets the user's voice command.
2. An appropriate response or action is executed based on the recognized command.

Main Success Scenario:

1. The user issues a voice command.
2. The system captures the voice input via the microphone.
3. The system processes the captured voice input using the voice recognition engine.
4. The system interprets the voice command accurately.
5. The system provides feedback to the user, either confirming the command or executing the requested action.

Extensions (Alternative Flows):

- **2a.** Microphone is not functioning:
 - 2a1. The system notifies the user of the microphone issue and prompts to check the device.
- **3a.** Camera is required for additional input but not functioning:
 - 3a1. The system notifies the user of the camera issue and prompts to check the device.
 - 3a2. The system offers to proceed with audio-only mode if camera input is optional.
- **4a.** Voice command is not recognized:
 - 4a1. The system requests the user to repeat the command.
 - 4a2. The system processes the repeated command.
 - 4a3. If the command is still not recognized, the system offers alternative input methods (e.g., text input).

Special Requirements:

1. The system must process voice commands with high accuracy, even with various accents and speech patterns.
2. The system should provide real-time feedback to ensure seamless interaction.

Technology and Data Variations List:

1. Voice recognition engine can be cloud-based (e.g., Google Cloud Speech-to-Text) or locally hosted.
2. Microphone can be built-in or external.
3. Camera can be built-in or external and used for optional visual input.

Frequency of Occurrence:

- Frequently, as it is the primary method for user interaction with the system.

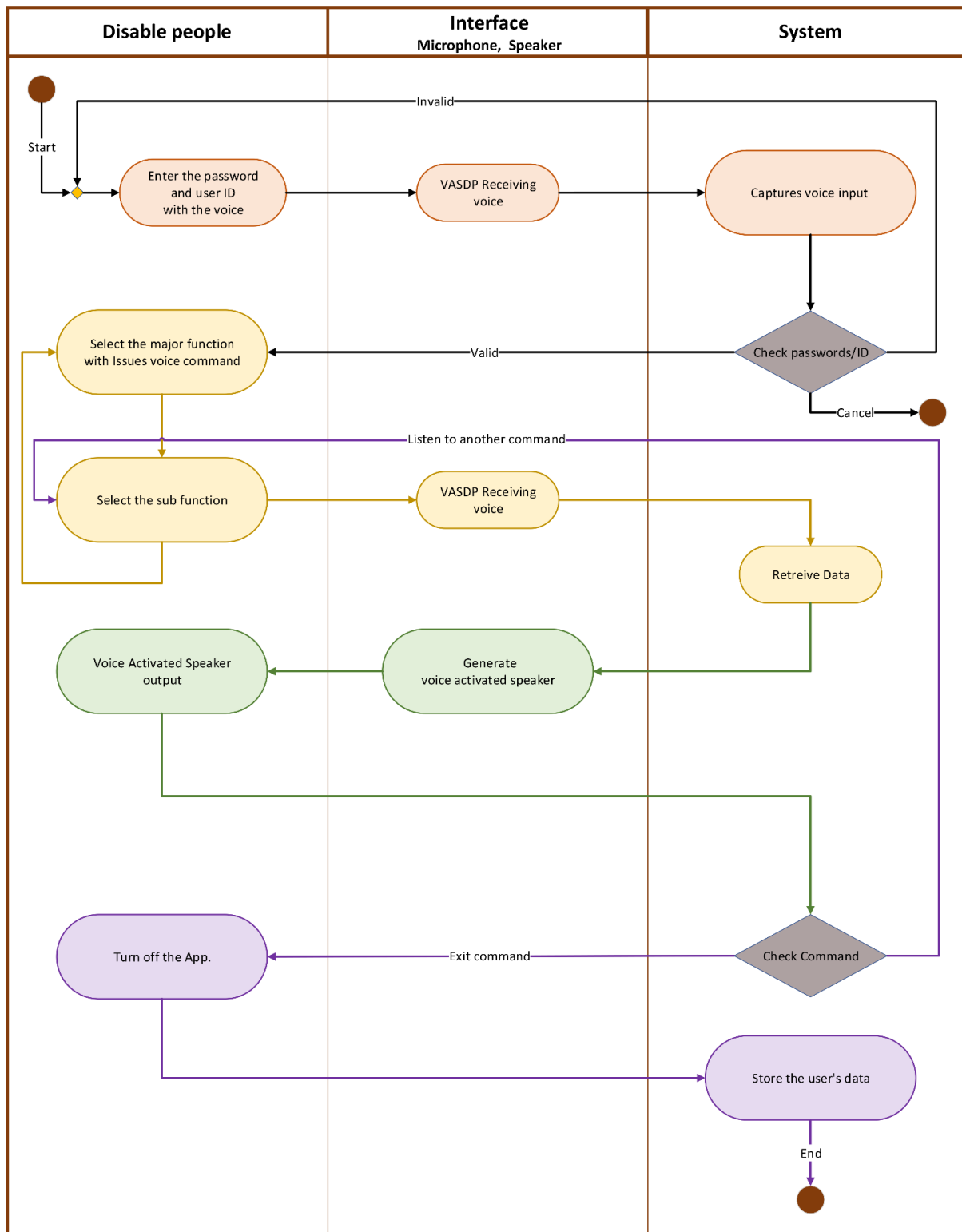
Open Issues:

1. Ensuring privacy and data security for voice and visual data.
2. Handling variations in speech, environmental noise, and lighting conditions for camera input.

4.3 SwimLane Activity Diagram

Below is a textual description of the swimlane activity diagram for the "Recognize-Voice Command" use case.

Swimlane Diagram



4.4 The Voice Activated Speaker for Disabled People System

Subsystems and Abstract Use Case

1. **User Management**
 - Create-User Profile
 - Manage-User Settings
 - **Abstract Use Case:** Log into System
2. **Voice Command Processing**
 - Recognize-Voice Command
 - Execute-Task
 - **Abstract Use Case:** Log into System
3. **Integration and Configuration**
 - Configure-Integration
 - Manage-Security
 - Ensure-Compatibility
 - **Abstract Use Case:** Configure System
 - **Abstract Use Case:** Log into System
4. **System Maintenance**
 - Monitor-System Health
 - Update-System
 - **Abstract Use Case:** Configure System
 - **Abstract Use Case:** Log into System

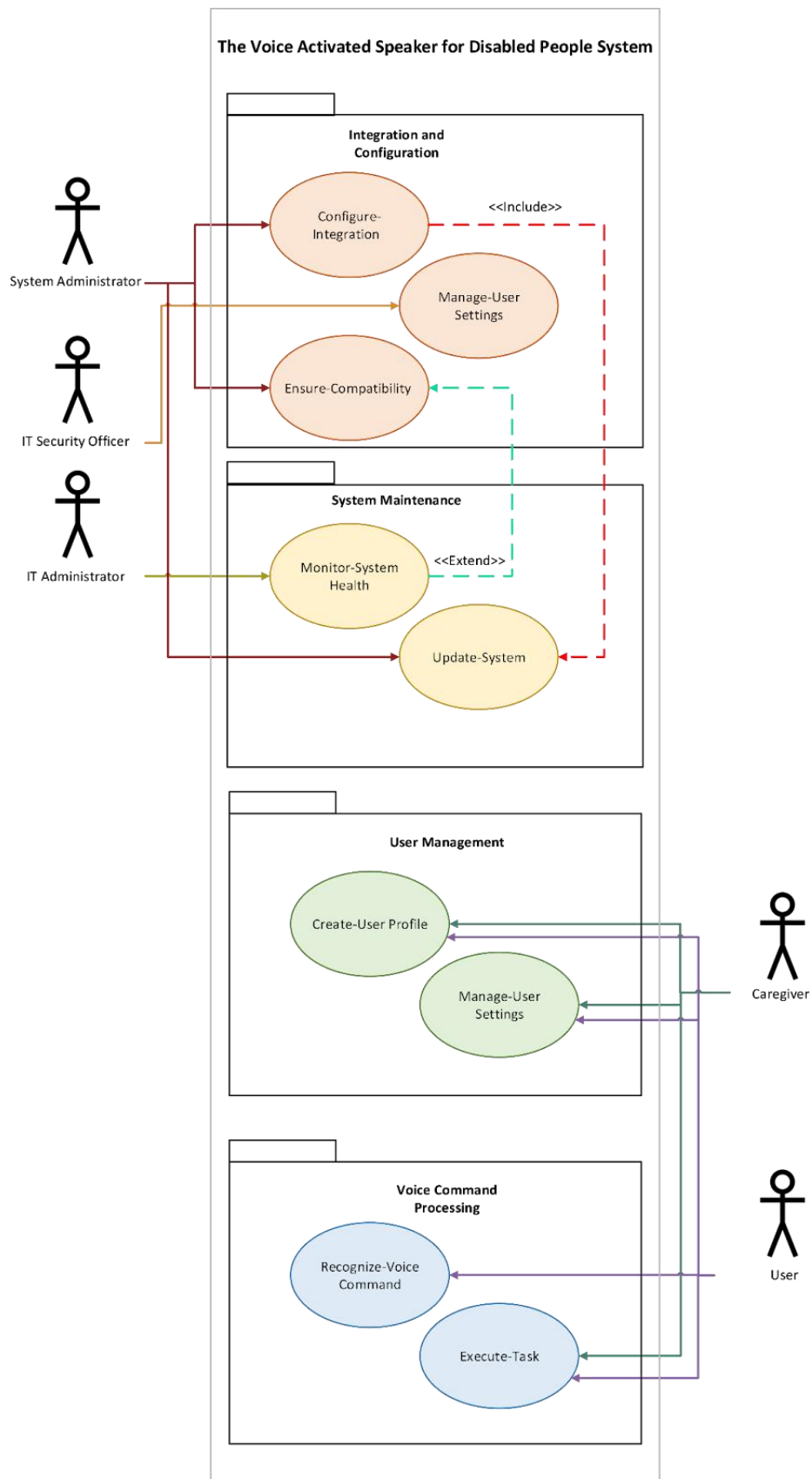
Actors

- User
- Caregiver
- System Administrator
- IT Security Officer
- IT Administrator

Connections

- **Log into System** should be connected to relevant use cases indicating the need for user authentication.
- **Configure System** should be connected to relevant use cases indicating system setup and configuration steps.

4.5 Use Case Diagram



Section 5

5.1 Nonfunctional Requirements list				
Requirement ID	Requirement Title	Short Description	Priority	Requester
NFR01	Performance	The system should process voice commands and respond within 2 seconds, regardless of the complexity of the command.	Expected	IT Administrator
NFR02	Security - Data Privacy	The system should ensure all user data is encrypted and protected to maintain privacy, especially for sensitive information.	High	IT Security Officer
NFR03	Reliability	The system should have an uptime of 99.9%, ensuring it is always available for users who rely on it for daily activities.	Normal/Medium	IT Administrator
NFR04	Compatibility	The system should be compatible with a wide range of assistive devices and smart home ecosystems.	Expected	Product Manager
NFR05	Scalability	The system should handle up to 1000	Normal/Medium	IT Administrator

		simultaneous users without performance degradation.		
NFR06	Energy Efficiency	The system should comply with energy efficiency standards to reduce power consumption, particularly important for users with limited mobility.	Low	Sustainability Officer
NFR07	Localization	The system should support multiple languages and regional accents, making it accessible to users worldwide..	Low	Product Manager
NFR08	User Interface	The system should be easy to update with new features and security patches, ensuring continuous improvement in accessibility.	High	IT Administrator

APPENDIX C

Stakeholder Register					
Stakeholder Name	Stakeholder Position	External/Internal	Stakeholder Contact Details	Operational/Executive	Interest (high, medium, low)
Davi V. T. Z.	Investor	External	dvolpita@my.centennialcollege.ca	Operational	High
Jaturaput J.	Marketing Officer	Internal	jjongsub@my.centennialcollege.ca	Operational	High
Rathapat A.	IT Administrator	Internal	rapichai@my.centennialcollege.ca	Operational	Medium
Kripa A.	Programmer	Internal	kachar10@my.centennialcollege.ca	Executive	Medium
Robert G.	Customer	External	robertg@abc.com	Operational	High
Daniel F.	IT Security Officer	Internal	danielf.@abc.com	Operational	Low

APPENDIX D

Interview Questions		
Question	Stakeholder Position	Answer

1. What is your budget for implementing and maintaining the VASDP? 2. What are your expected financial returns from VASDP in the next five years? 3. How much risk are you willing to accept in VASDP? 4. What level of involvement do you want to have in the development of the VASDP? 5. Are there milestones you expect to be met?	Investor	1. An initial investment of \$500,000 with a yearly maintenance budget of \$100,000. 2. A 20% annual return on the investment, achieving break-even in less than five years. 3. Moderate risk, a risk mitigation plan must be in place. 4. To be involved in major strategic decisions, with regular updates. 5. Key milestones include completing the prototype in 12 months and achieving 10000 users within the first-year post-launch.
1. How frequently do you expect updates or new features to be released for the VASDP? 2. What key demographics are you targeting? 3. What channels do you think will be most effective in promoting VASDP? 4. How do you plan to measure the success of the marketing campaigns for VASDP? 5. How do you plan to collect and utilize user feedback?	Marketing Officer	1. Annually updates with significant feature releases. 2. Adults with disabilities, caregivers and rehabilitation centers. 3. Social media, online disability forums and partnerships with disability advocacy groups. 4. User-friendly interface and seamless integration with existing tech. 5. User surveys, focus groups, and direct feedback channels within the app to collect insights, which will be

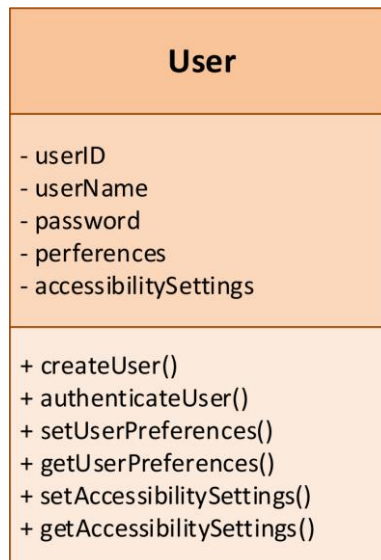
		analyzed regularly for updates and new features.
1. What security measures do you recommend for the VASDP to protect user data? 2. How will you handle potential downtime or technical issues? 3. User training and documentation will be provided? 4. What are your expectations regarding the compatibility of VASDP with other software and hardware? 5. What are the potential challenges in VASDP integration into the IT ecosystem?	IT Administrator	1. Implementation of end-to-end encryption, regular security audits, and strict access control. 2. With a robust incident response plan, with backup systems and a dedicated 24/7 support team to address any issues. 3. Yes, comprehensive user manuals and video tutorials will be provided for both end-users and staff. 4. It should be compatible with current operating systems, database management systems, and IoT devices. 5. Possible challenges are data migration issues, ensuring compatibility with legacy systems, and maintaining security.
1. What programming languages and frameworks do you recommend for developing the VASDP? 2. What tools do you suggest for voice recognition and natural language processing in VASDP? 3. How would you	Programmer	1. React Native for the mobile app interface, and Node.js for the server-side operations. 2. Google's Speech-To-Text API, TensorFlow and Amazon Transcribe. 3. By using microservices architecture, load balancing, and

ensure the scalability and performance of VASDP? 4. What development methodologies would you propose for VASDP? 5. What testing strategies do you recommend for VASDP to ensure high quality?		cloud-based solutions from AWS. 4. Agile methodology with iterative development cycles. 5. Unit testing, integration testing and user acceptance testing.
--	--	--

APPENDIX E

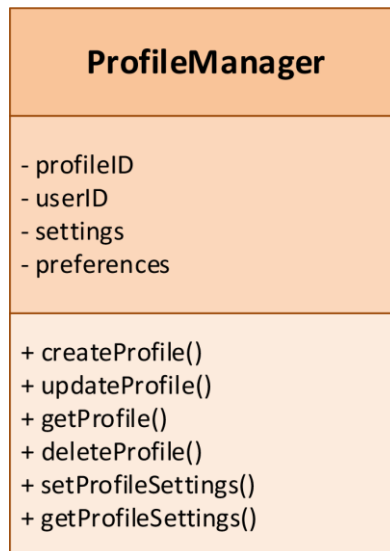
CLASS DIAGRAM

1. User



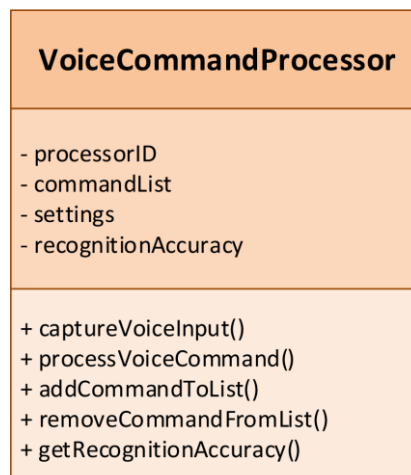
- **Attributes:**
 - **userID**: Unique identifier for the user.
 - **userName**: Name of the user.
 - **password**: User's password for authentication.
 - **preferences**: User-specific preferences.
 - **accessibilitySettings**: Settings tailored to the user's accessibility needs.
- **Methods:**
 - **createUser()**: Create a new user profile.
 - **authenticateUser()**: Authenticate the user.
 - **setUserPreferences()**: Set user preferences.
 - **getUserPreferences()**: Retrieve user preferences.
 - **setAccessibilitySettings()**: Configure accessibility settings.
 - **getAccessibilitySettings()**: Retrieve accessibility settings.
- **Relationships:**
 - "is managed by" **ProfileManager**
 - "issues commands to" **VoiceCommandProcessor**
 - "configures" **AccessibilitySettings**
 - "provides data to" **SecurityManager**

2. ProfileManager



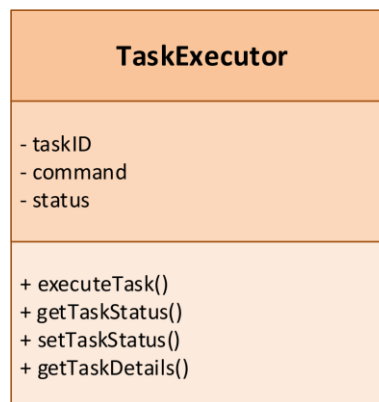
- **Attributes:**
 - **profileID:** Unique identifier for the profile.
 - **userID:** Identifier linking the profile to a user.
 - **settings:** Profile-specific settings.
 - **preferences:** Profile-specific preferences.
- **Methods:**
 - **createProfile():** Create a new profile.
 - **updateProfile():** Update an existing profile.
 - **getProfile():** Retrieve profile information.
 - **deleteProfile():** Delete a profile.
 - **setProfileSettings():** Configure profile settings.
 - **getProfileSettings():** Retrieve profile settings.
- **Relationships:**
 - "manages" **User**

3. VoiceCommandProcessor



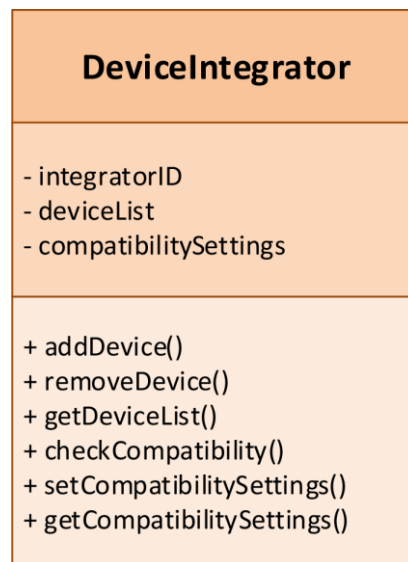
- **Attributes:**
 - **processorID**: Unique identifier for the processor.
 - **commandList**: List of recognized commands.
 - **recognitionAccuracy**: Accuracy rate of voice recognition.
- **Methods:**
 - **captureVoiceInput()**: Capture voice input from the user.
 - **processVoiceCommand()**: Process the captured voice command.
 - **addCommandToList()**: Add a new command to the list.
 - **removeCommandFromList()**: Remove a command from the list.
 - **getRecognitionAccuracy()**: Retrieve the recognition accuracy rate.
- **Relationships:**
 - "processes commands for" **User**
 - "sends tasks to" **TaskExecutor**

4. TaskExecutor



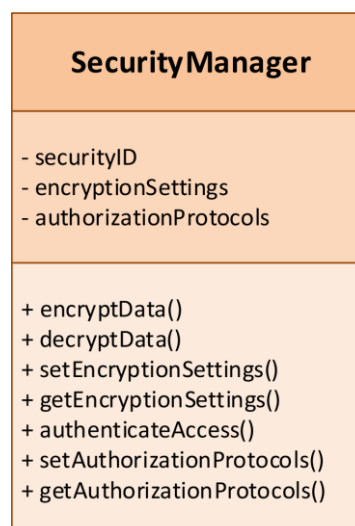
- **Attributes:**
 - **taskID**: Unique identifier for the task.
 - **command**: Command to be executed.
 - **status**: Status of the task.
- **Methods:**
 - **executeTask()**: Execute the given task.
 - **getTaskStatus()**: Retrieve the status of the task.
 - **setTaskStatus()**: Update the status of the task.
 - **getTaskDetails()**: Retrieve details of the task.
- **Relationships:**
 - "executes tasks for" **VoiceCommandProcessor**
 - "uses devices from" **DeviceIntegrator**
 - "is monitored by" **SystemHealthMonitor**

5. DeviceIntegrator



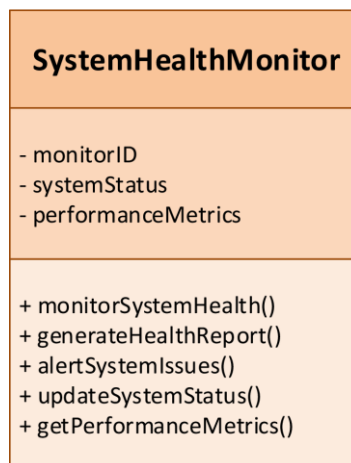
- **Attributes:**
 - **integratorID**: Unique identifier for the integrator.
 - **deviceList**: List of integrated devices.
 - **compatibilitySettings**: Compatibility settings for devices.
- **Methods:**
 - **addDevice()**: Add a new device to the system.
 - **removeDevice()**: Remove a device from the system.
 - **getDeviceList()**: Retrieve the list of devices.
 - **checkCompatibility()**: Check the compatibility of devices.
 - **setCompatibilitySettings()**: Configure compatibility settings.
 - **getCompatibilitySettings()**: Retrieve compatibility settings.
- **Relationships:**
 - "provides devices to" **TaskExecutor**

6. SecurityManager



- **Attributes:**
 - **securityID**: Unique identifier for the security manager.
 - **encryptionSettings**: Settings for data encryption.
 - **authorizationProtocols**: Protocols for user authorization.
- **Methods:**
 - **encryptData()**: Encrypt data for security.
 - **decryptData()**: Decrypt data for use.
 - **setEncryptionSettings()**: Configure encryption settings.
 - **getEncryptionSettings()**: Retrieve encryption settings.
 - **authenticateAccess()**: Authenticate user access.
 - **setAuthorizationProtocols()**: Configure authorization protocols.
 - **getAuthorizationProtocols()**: Retrieve authorization protocols.
- **Relationships:**
 - "secures data for" User

7. SystemHealthMonitor



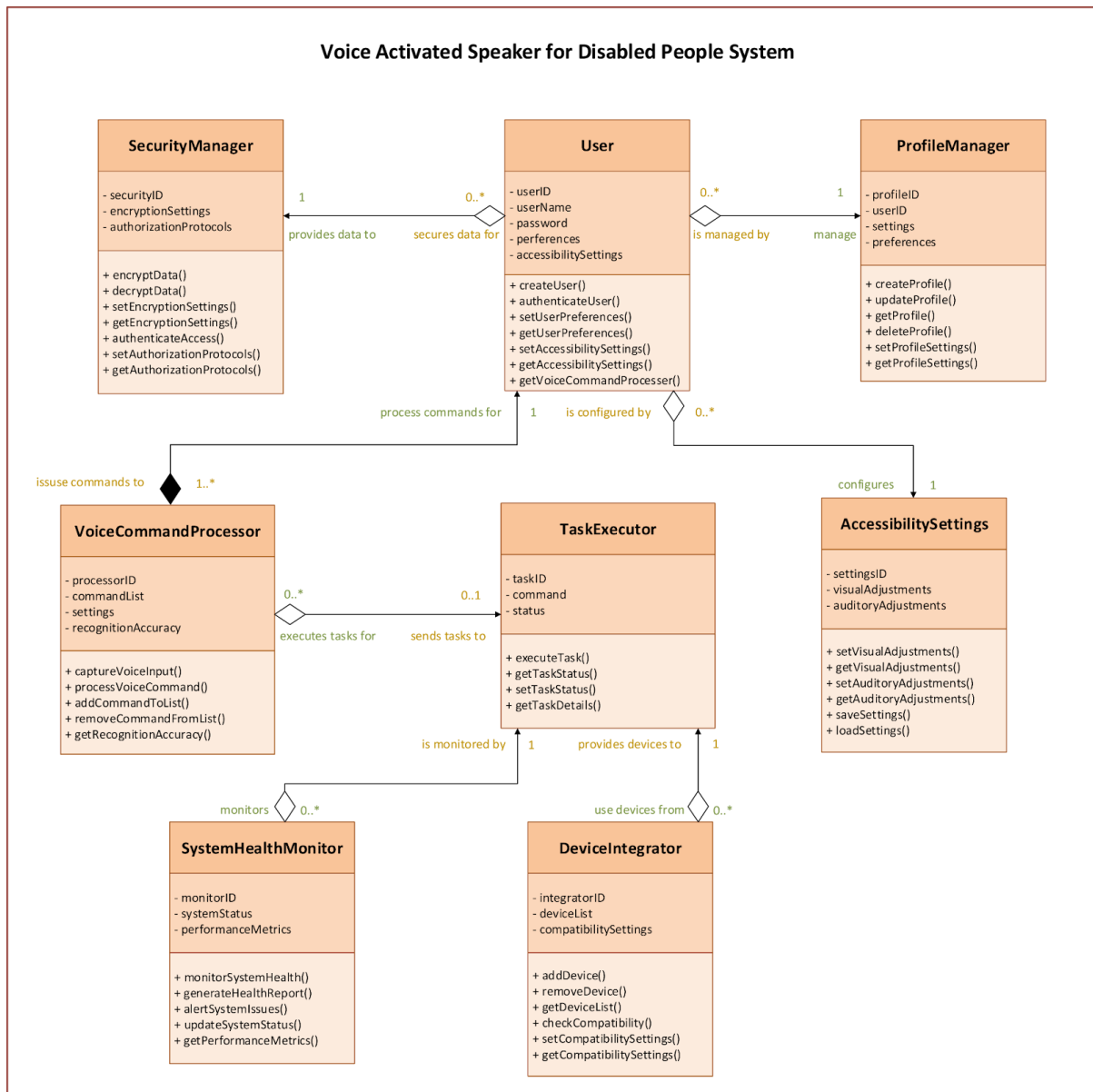
- **Attributes:**
 - **monitorID**: Unique identifier for the monitor.
 - **systemStatus**: Status of the system.
 - **performanceMetrics**: Performance metrics of the system.
- **Methods:**
 - **monitorSystemHealth()**: Monitor the health of the system.
 - **generateHealthReport()**: Generate a report on system health.
 - **alertSystemIssues()**: Alert when issues are detected.
 - **updateSystemStatus()**: Update the status of the system.
 - **getPerformanceMetrics()**: Retrieve performance metrics.
- **Relationships:**
 - "monitors" TaskExecutor

8. AccessibilitySettings

AccessibilitySettings
- settingsID - visualAdjustments - auditoryAdjustments
+ setVisualAdjustments() + getVisualAdjustments() + setAuditoryAdjustments() + getAuditoryAdjustments() + saveSettings() + loadSettings()

- **Attributes:**
 - **settingsID**: Unique identifier for the settings.
 - **visualAdjustments**: Adjustments for visual accessibility.
 - **auditoryAdjustments**: Adjustments for auditory accessibility.
- **Methods:**
 - **setVisualAdjustments()**: Configure visual adjustments.
 - **getVisualAdjustments()**: Retrieve visual adjustments.
 - **setAuditoryAdjustments()**: Configure auditory adjustments.
 - **getAuditoryAdjustments()**: Retrieve auditory adjustments.
 - **saveSettings()**: Save the current settings.
 - **loadSettings()**: Load the saved settings.
- **Relationships:**
 - "is configured by" **User**

UML static structure diagram:



Class Relationships

User creates and manages ProfileManager.

- The **User** is responsible for creating and managing their profile through the **ProfileManager**.

User interacts with VoiceCommandProcessor to issue voice commands.

- The **User** issues voice commands that are processed by the **VoiceCommandProcessor**.

VoiceCommandProcessor communicates with TaskExecutor to execute tasks.

- The **VoiceCommandProcessor** processes the user's commands and communicates with the **TaskExecutor** to perform the necessary tasks.

DeviceIntegrator ensures compatibility and integration with external devices for VoiceCommandProcessor and TaskExecutor.

- The **DeviceIntegrator** integrates and ensures the compatibility of external devices with the **VoiceCommandProcessor** and **TaskExecutor**.

SecurityManager ensures the security of data handled by ProfileManager and User.

- The **SecurityManager** handles the security of data for both the **ProfileManager** and **User**, including encryption, authentication, and authorization.

SystemHealthMonitor keeps track of system health and interacts with TaskExecutor and SecurityManager for updates and alerts.

- The **SystemHealthMonitor** monitors the overall system health, providing updates and alerts to the **TaskExecutor** and **SecurityManager**.

AccessibilitySettings are managed by User and ProfileManager.

- The **AccessibilitySettings** are configured and managed by the **User** and **ProfileManager** to ensure the system meets the user's accessibility needs.

CRC Cards for "Voice Activated Speaker for Disabled People"

1. Class: User

- **Responsibilities:**
 - Create and manage user profiles
 - Set preferences and settings for voice commands
 - Authenticate user access
- **Collaborators:**
 - ProfileManager
 - VoiceCommandProcessor

CRC card (front side)	
Class: User	
Description: Represents an individual using the system, responsible for interacting with the voice-activated speaker.	
Responsibilities:	Collaborators:
Create and manage user profiles	ProfileManager
Set preferences and settings for voice commands	VoiceCommandProcessor
Authenticate user access	

CRC card (back side)
Attributes:
1. userID : Integer (private) 2. userName : String (private) 3. password : String (private) 4. preferences : String (private)

5. accessibilitySettings : String (private)

2. Class: ProfileManager

- **Responsibilities:**
 - Manage user profiles and settings
 - Store and retrieve user preferences
- **Collaborators:**
 - User
 - VoiceCommandProcessor

CRC card (front side)

Class: ProfileManager

Description: Manages user profiles and settings within the system.

Responsibilities:

Collaborators:

Manage user profiles and settings

User

Store and retrieve user preferences

VoiceCommandProcessor

CRC card (back side)

Attributes:

1. profileID : Integer (private)
2. userID : Integer (private)
3. settings : String (private)
4. preferences : String (private)

3. Class: VoiceCommandProcessor

- **Responsibilities:**
 - Recognize and process voice commands
 - Translate voice commands into system actions
- **Collaborators:**
 - User
 - TaskExecutor
 - ProfileManager

CRC card (front side)	
Class: VoiceCommandProcessor	
Description: Manages user profiles and settings within the system.	
Responsibilities:	Collaborators:
Recognize and process voice commands	User
Translate voice commands into system actions	TaskExecutor
	ProfileManager

CRC card (back side)
Attributes:

1. processorID : Integer (private)
2. commandList : List<String> (private)
3. recognitionAccuracy : Float (private)

4. Class: TaskExecutor

- **Responsibilities:**
 - Execute tasks based on recognized voice commands
 - Provide feedback on task completion
- **Collaborators:**
 - VoiceCommandProcessor
 - DeviceIntegrator

CRC card (front side)

Class: TaskExecutor

Description: Executes tasks based on processed voice commands.

Responsibilities:

Execute tasks based on recognized voice commands

Provide feedback on task completion

Collaborators:

VoiceCommandProcessor

DeviceIntegrator

CRC card (back side)

Attributes:

1. taskID : Integer (private)
2. command : String (private)

3. status : String (private)

5. Class: DeviceIntegrator

- **Responsibilities:**
 - Integrate with external devices and systems
 - Manage connections to smart home devices and other assistive technologies
- **Collaborators:**
 - VoiceCommandProcessor
 - TaskExecutor

CRC card (front side)

Class: DeviceIntegrator

Description: Integrates and manages external devices with the system.

Responsibilities:

Integrate with external devices and systems

Manage connections to smart home devices and other assistive technologies

Collaborators:

VoiceCommandProcessor

TaskExecutor

CRC card (back side)

Attributes:

1. integratorID : Integer (private)
2. deviceList : List<String> (private)
3. compatibilitySettings : String (private)

6. Class: SecurityManager

- **Responsibilities:**
 - Ensure data privacy and security
 - Manage authentication and authorization protocols
- **Collaborators:**
 - User
 - ProfileManager

CRC card (front side)**Class: SecurityManager**

Description: Manages the security aspects of the system, ensuring data privacy and secure access.

Responsibilities:**Collaborators:**

Ensure data privacy and security

User

Manage authentication and authorization protocols

ProfileManager

CRC card (back side)
Attributes:
1. monitorID : Integer (private) 2. systemStatus : String (private) 3. performanceMetrics : String (private)

7. Class: SystemHealthMonitor

- **Responsibilities:**
 - Monitor system performance and health
 - Alert on system issues and manage updates
- **Collaborators:**
 - TaskExecutor
 - SecurityManager

CRC card (front side)	
Class: SystemHealthMonitor	
Description: Monitors the health and performance of the system, providing alerts and updates.	
Responsibilities:	Collaborators:
Monitor system performance and health	TaskExecutor
Alert on system issues and manage updates	SecurityManager

CRC card (back side)
Attributes:
1. monitorID : Integer (private) 2. systemStatus : String (private) 3. performanceMetrics : String (private)

8. Class: AccessibilitySettings

- **Responsibilities:**
 - Manage accessibility settings for different types of disabilities
 - Provide options for visual and auditory adjustments
- **Collaborators:**
 - User
 - ProfileManager

CRC card (front side)	
Class: AccessibilitySettings	
Description: Manages the accessibility settings to cater to different user disabilities.	
Responsibilities:	Collaborators:
Manage accessibility settings for different types of disabilities	User
Provide options for visual and auditory adjustments	ProfileManager

CRC card (back side)
Attributes:
1. settingsID : Integer (private) 2. visualAdjustments : String (private) 3. auditoryAdjustments : String (private)

Updated First Cut Domain Class Diagram

a. Class Attribute Descriptions and Visibilities

1. User

a. Attributes:

- i. userID : Integer (private)
- ii. userName : String (private)
- iii. password : String (private)
- iv. preferences : String (private)
- v. accessibilitySettings : String (private)

b. Methods:

- i. + createUser() : void
- ii. + authenticateUser(password: String) : boolean
- iii. + setUserPreferences(preferences: String) : void
- iv. + getUserPreferences() : String
- v. + setAccessibilitySettings(settings: String) : void
- vi. + getAccessibilitySettings() : String

2. ProfileManager

a. Attributes:

- i. profileID : Integer (private)
- ii. userID : Integer (private)
- iii. settings : String (private)
- iv. preferences : String (private)

b. Methods:

- i. + createProfile(userID: Integer) : void
- ii. + updateProfile(profileID: Integer, settings: String) : void
- iii. + getProfile(profileID: Integer) : String
- iv. + deleteProfile(profileID: Integer) : void
- v. + setProfileSettings(profileID: Integer, settings: String) : void
- vi. + getProfileSettings(profileID: Integer) : String

3. VoiceCommandProcessor**a. Attributes:**

- i. processorID : Integer (private)
- ii. commandList : List<String> (private)
- iii. recognitionAccuracy : Float (private)

b. Methods:

- i. + captureVoiceInput() : String
- ii. + processVoiceCommand(command: String) : void
- iii. + addCommandToList(command: String) : void
- iv. + removeCommandFromList(command: String) : void
- v. + getRecognitionAccuracy() : Float

4. TaskExecutor**a. Attributes:**

- i. taskID : Integer (private)
- ii. command : String (private)
- iii. status : String (private)

b. Methods:

- i. + executeTask(command: String) : void
- ii. + getTaskStatus(taskID: Integer) : String
- iii. + setTaskStatus(taskID: Integer, status: String) : void
- iv. + getTaskDetails(taskID: Integer) : String

5. DeviceIntegrator**a. Attributes:**

- i. integratorID : Integer (private)
- ii. deviceList : List<String> (private)
- iii. compatibilitySettings : String (private)

b. Methods:

- i. + addDevice(device: String) : void
- ii. + removeDevice(deviceID: Integer) : void
- iii. + getDeviceList() : List<String>
- iv. + checkCompatibility(device: String) : boolean
- v. + setCompatibilitySettings(settings: String) : void
- vi. + getCompatibilitySettings() : String

6. SecurityManager

a. Attributes:

- i. securityID : Integer (private)
- ii. encryptionSettings : String (private)
- iii. authorizationProtocols : String (private)

b. Methods:

- i. + encryptData(data: String) : String
- ii. + decryptData(encryptedData: String) : String
- iii. + setEncryptionSettings(settings: String) : void
- iv. + getEncryptionSettings() : String
- v. + authenticateAccess(userID: Integer) : boolean
- vi. + setAuthorizationProtocols(protocols: String) : void
- vii. + getAuthorizationProtocols() : String

7. SystemHealthMonitor

a. Attributes:

- i. monitorID : Integer (private)
- ii. systemStatus : String (private)
- iii. performanceMetrics : String (private)

b. Methods:

- i. + monitorSystemHealth() : void
- ii. + generateHealthReport() : String
- iii. + alertSystemIssues(issue: String) : void
- iv. + updateSystemStatus(status: String) : void
- v. + getPerformanceMetrics() : String

8. AccessibilitySettings

a. Attributes:

- i. settingsID : Integer (private)
- ii. visualAdjustments : String (private)
- iii. auditoryAdjustments : String (private)

b. Methods:

- i. + setVisualAdjustments(adjustments: String) : void
- ii. + getVisualAdjustments() : String
- iii. + setAuditoryAdjustments(adjustments: String) : void
- iv. + getAuditoryAdjustments() : String
- v. + saveSettings() : void
- vi. + loadSettings() : String

b. Relationships and Signature Methods

1. User creates and manages ProfileManager.

- a. User class has a method createUser().

- b. ProfileManager has methods createProfile(userID: Integer), updateProfile(profileID: Integer, settings: String).
- 2. **User interacts with VoiceCommandProcessor to issue voice commands.**
 - a. User class uses method setUserPreferences(preferences: String).
 - b. VoiceCommandProcessor has methods captureVoiceInput(), processVoiceCommand(command: String).
- 3. **VoiceCommandProcessor communicates with TaskExecutor to execute tasks.**
 - a. VoiceCommandProcessor uses method processVoiceCommand(command: String).
 - b. TaskExecutor has method executeTask(command: String).
- 4. **DeviceIntegrator ensures compatibility and integration with external devices for VoiceCommandProcessor and TaskExecutor.**
 - a. DeviceIntegrator has methods addDevice(device: String), checkCompatibility(device: String).
- 5. **SecurityManager ensures the security of data handled by ProfileManager and User.**
 - a. SecurityManager has methods encryptData(data: String), authenticateAccess(userID: Integer).
- 6. **SystemHealthMonitor keeps track of system health and interacts with TaskExecutor and SecurityManager for updates and alerts.**
 - a. SystemHealthMonitor has methods monitorSystemHealth(), alertSystemIssues(issue: String).
- 7. **AccessibilitySettings are managed by User and ProfileManager.**
 - a. AccessibilitySettings has methods setVisualAdjustments(adjustments: String), saveSettings().

Abstract State Machine Diagram for TaskExecutor

The **TaskExecutor** manages the execution of commands and handles the transition between various states during task execution. The state machine diagram consists of the following states, transitions, and actions:

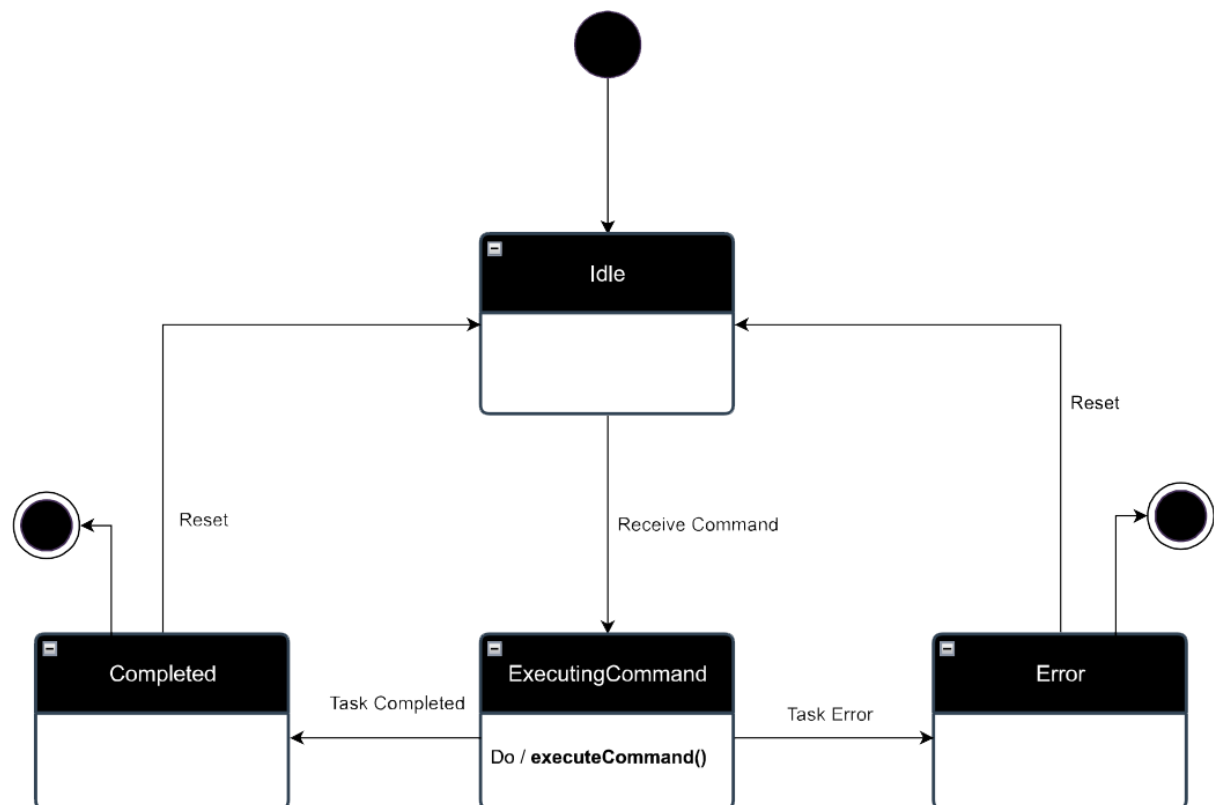
- 1. **Idle State:**
 - **Entry:** The system begins in an idle state, awaiting commands to be executed.
 - **Transition:** When a command is received (**Receive Command**), it transitions to the ExecutingCommand state.
- 2. **ExecutingCommand State:**
 - **Behavior:** This state is responsible for executing the received command.
 - **Action:** The **executeCommand()** action is performed to carry out the task.
 - **Transitions:**

- If the task is completed successfully (**Task Completed**), it transitions to the Completed state.
 - If an error occurs during execution (**Task Error**), it transitions to the Error state.
3. **Completed State:**
- **Behavior:** Marks the successful completion of the command execution.
 - **Transition:** Returns to the Idle state upon reset (**Reset**).
4. **Error State:**
- **Behavior:** Handles errors encountered during task execution.
 - **Transition:** After addressing the error or upon reset (**Reset**), it transitions back to the Idle state.

Summary:

This state machine diagram captures the dynamic behavior of the **TaskExecutor**, focusing on its capability to execute commands and manage errors. The transitions and actions ensure efficient processing and return the system to an idle state once tasks are completed or errors are resolved.

TaskExecutor system



Abstract State Machine Diagram for VoiceCommandProcessor

The **VoiceCommandProcessor** is designed to handle voice input from the user, process commands, and execute them. It consists of the following key states, transitions, and actions:

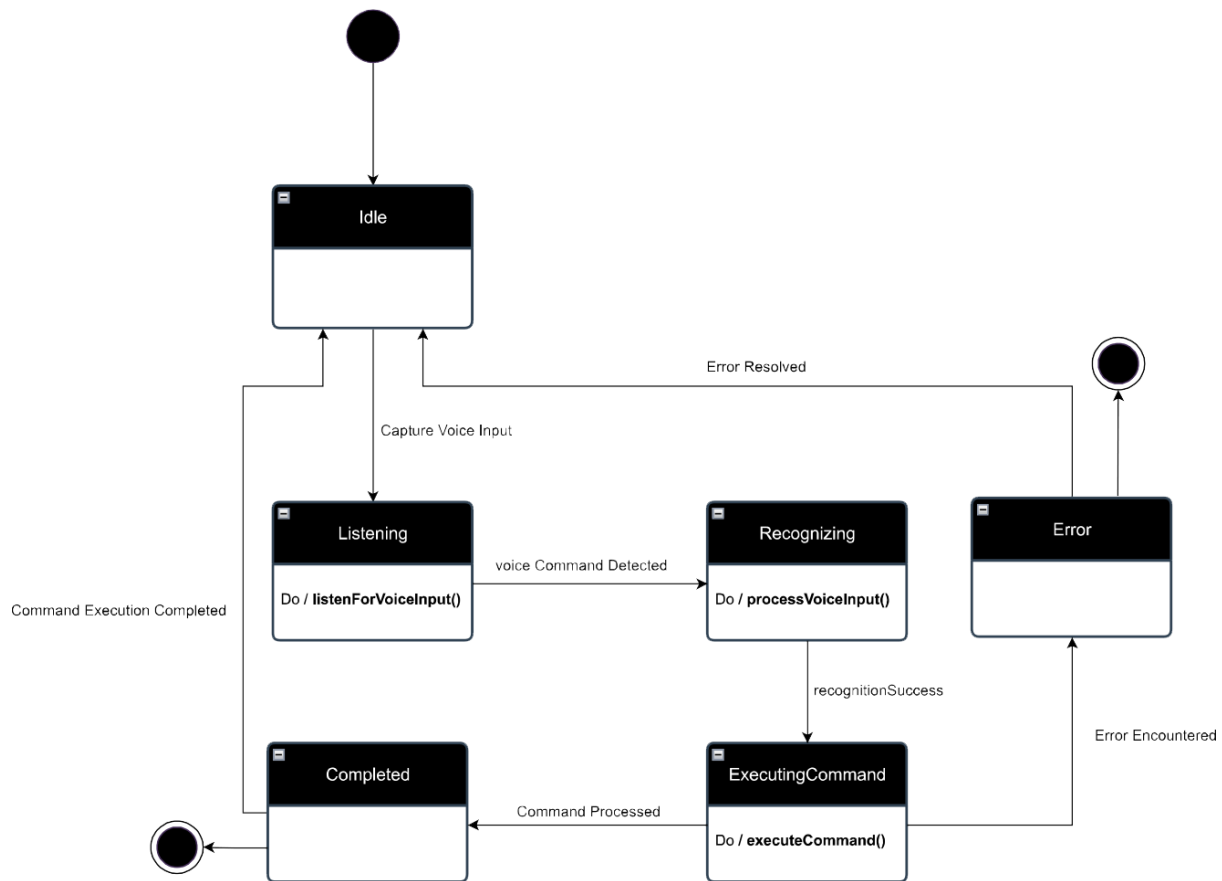
1. **Idle State:**
 - **Entry:** The system starts in an idle state, ready to capture voice input.
 - **Transition:** Upon receiving a command from the user (**Capture Voice Input**), it transitions to the Listening state.
2. **Listening State:**
 - **Behavior:** In this state, the system is actively listening for voice commands.
 - **Action:** The **listenForVoiceInput()** action is performed to capture the user's voice.
 - **Transition:** Once a voice command is detected (**voiceCommandDetected**), it moves to the Recognizing state.
3. **Recognizing State:**
 - **Behavior:** The system processes the captured voice input to recognize the command.
 - **Action:** The **processVoiceInput()** action is executed to interpret the command.
 - **Transitions:**
 - If the command is successfully recognized (**recognitionSuccess**), it transitions to the ExecutingCommand state.
 - If an error is encountered (**Error Encountered**), it transitions to the Error state.
4. **ExecutingCommand State:**
 - **Behavior:** Executes the recognized command.
 - **Action:** The **executeCommand()** action is carried out to perform the required task.
 - **Transition:** Upon successful execution of the command (**Command Processed**), it transitions to the Completed state.
5. **Completed State:**
 - **Behavior:** Marks the successful completion of command execution.
 - **Transition:** Returns to the Idle state (**Command Execution Completed**) after the command execution is completed.
6. **Error State:**
 - **Behavior:** Handles any errors encountered during recognition or execution.
 - **Transition:** After resolving the error (**Error Resolved**), it transitions back to the Idle state.

Summary:

This state machine diagram captures the dynamic behavior of the **VoiceCommandProcessor**, emphasizing its ability to listen, recognize, and execute voice commands while handling errors gracefully. The transitions and actions ensure that the system

remains responsive and efficient, returning to the idle state once tasks are completed or errors are resolved.

VoiceCommandProcessor system



Abstract Sequence Diagram for Voice Command Processing

Actors:

- **User**: Represents the disabled person interacting with the system.
- **Interface**: Handles the voice input from the user.
- **VoiceCommandProcessor (VCP)**: Processes voice commands.
- **Database (DB)**: Validates user credentials.

Main Processes:

1. User Authentication:

- The **User** enters a password and user ID via voice to the **Interface**.
- The **Interface** transmits the voice input to the **VoiceCommandProcessor**.
- **VCP** captures the voice input and sends a validation request to the **Database**.
- The **Database** checks the credentials and returns a validation response to the **VCP**.
- If the credentials are valid, **VCP** provides feedback to the **User** indicating access is granted.
- If the credentials are invalid, **VCP** provides feedback indicating access is denied.

2. Command Listening and Execution:

- **VCP** enters a loop to listen for commands from the **User**.
 - The **User** speaks a command to the **Interface**.
 - The **Interface** transmits the command to the **VCP**.
 - **VCP** recognizes the command and provides feedback to the **User**.
 - **VCP** executes the recognized command.
 - If the command execution is successful, **VCP** provides success feedback to the **User**.
 - If the command execution fails, **VCP** provides feedback indicating the failure.
3. **Feedback Loop:**
- If a command is not recognized, **VCP** prompts the **User** to try again.
 - The loop continues until the **User** successfully executes a command or exits.

Alternative Flows:

- **Command Recognition Failure:** If a command is not recognized, **VCP** prompts the **User** to repeat the command.
- **Authentication Failure:** If user authentication fails, access is denied, and the sequence ends.

Sequence diagram: Voice Command Processing

